Laborationsrapport

Projektuppgift
DT211G, Frontend-baserad webbutveckling

Ludvig Rosenqvist, luro2501@student.miun.se
VT, 2026

Mittuniversitetet
MID SWEDEN UNIVERSITY

1 Sammanfattning

Denna rapport innehåller konstruktionen från början till start av en webbplats vars syfte är att presentera F1 kalendern 2026 med hjälp av API anrop. Utöver det har en karttjänst implementerats för att skapa mashup-webbsida.

Webbplatsen är en single-page application som är byggd med hjälp av frontend-verktyget Vite. Detta har bidragit till en arbetsprocess som automatiserar stora delar av projektet.

Det färdiga resultatet är en webbplats som F1-följare kan använda för att få information om kalendern eller få vägbeskrivning till ett race med hjälp av Google Maps.

2 Innehållsförteckning

1 Sammanfattning ... 1
2 Innehållsförteckning ... 2
3 Inledning ... 3
3.1 Syfte ... 4
3.2 Problemformulering ... 4
4 Teori ... 4
4.1 Mashup ... 5
4.2 Application Programming Interface (API) ... 5
4.3 Kartor med Leaflet ... 5
4.4 Utvecklingsmiljö ... 5
4.5 JSDoc ... 5
5 Metod ... 5
5.1 Datainsamling och API-hantering ... 6
5.2 Kartor ... 6
5.3 Sök- och filtreringslogik ... 6
5.4 Implementering av ljust och mörkt läge ... 6
6 Konstruktion ... 6
6.1 Automatiserat arbetsflöde med Vite ... 7
6.2 Modulär filstruktur ... 7
6.3 Layout och styling med SCSS Partials ... 8
6.4 Filtrering och sökning ... 8
6.5 Animationer och interaktioner ... 9
7 Resultat ... 10
7.1 Webbsidans gränssnitt ... 11
7.2 Kartfunktioner ... 11
7.3 Sökning och filtrering i realtid ... 11
7.4 Visuella teman (Light/Dark mode) ... 12
7.5 Animationer och interaktioner ... 12
8 Slutsatser ... 12
8.1 Reflektion kring resultat och arbetsflöde ... 13
8.2 Lärdomar och utveckling ... 13
8.3 Framtida utveckling ... 13
9 Källförteckning ... 13

3 Inledning

3.1 Syfte

Detta projekt presenterar en webbsida som kombinerar två externa API:er (mashup) för att skapa ett schema och reseplanerare för Formel 1-entusiaster. Syftet är att presentera ett komplett tävlingsschema med tider, datum och platser, kompletterat med interaktiva kartor som visar banans exakta koordinater och tillhörande vägbeskrivningar genom att integrera Jolpica F1 API med OpenStreetMap API.

3.2 Problemformulering

För att uppnå webbsidans syfte har följande problemställningar identifierats:

- **Extern API:** Hämta data flexibelt med externa API:er.
- **Användarvänlighet:** Presentera sidans innehåll på ett användarvänligt och tillgängligt sätt.
- **Kartor:** Implementering av interaktiva kartor och ruttplanering för varje race.
- **Sökbarhet:** Sökfunktion för att filtrera lopp baserat på land och namn.
- **Ljust/Mörkt läge:** Automatisk anpassning efter användarens systeminställningar för ljust eller mörkt läge (dark mode), med möjlighet till manuell styrning.

4 Teori

4.1 Mashup

En *mashup* är en applikation eller en webbsida som kombinerar data från mer än en källa för att skapa ett nytt och unikt verktyg. Detta kan bidra till att slutanvändaren endast behöver använda en applikation eller webbsida istället för flera. [1]

4.2 Application Programming Interface (API)

Ett **API** fungerar är ett gränssnitt för att föra över data mellan två punkter. Standardformaten för datan är JSON eller XML men andra typer kan förekomma. Inom webbutveckling skickas detta via HTTP-protokollet. [2]

- **Jolpica F1 API:** Ett API som levererar data om Formel 1-mästerskap, såsom tävlingskalendrar och resultat. Jolpica är en ersättare till det föregående API:et Ergast. [3]

- **OpenStreetMap (OSM):** Ett API för att extrahera geografisk information, med hjälp av andra tjänster är det möjligt att generera kartbilder (tiles) för användning på webben. [4]

4.3 Kartor med Leaflet

För att använda geografisk data på en webbsida krävs ett specifikt JavaScript bibliotek till exempel **Leaflet**. Leaflet fungerar ungefär som en canvas att rita upp information från källor som OpenStreetMap. Leaflet möjliggör avancerad funktionalitet såsom interaktiva markörer och zoomfunktioner. [5]

4.4 Utvecklingsmiljö

- **npm (Node Package Manager):** En pakethanterare som används för att installera externa bibliotek och moduler. Genom att använda npm får man tillgång till delad kod och man kan säkerställa att alla utvecklare använder samma version av biblioteken. [6]

- **Vite:** Ett modernt frontend-verktyg som används för att paketera och optimera källkod. Vite erbjuder en utvecklingsmiljö där det monotona arbetet automatiseras. Exempelvis minimering, konkatenering och översättning mellan olika syntaxer. [7]

4.5 JSDoc

JSDoc är en standard för att dokumentera JavaScript-kod direkt i källfilerna via särskilda kommentarer. Genom att definiera parametrar och returtyper hjälper JSDoc utvecklaren att förstå funktioners syfte och förväntad data. I moderna kodredigerare som Visual Studio Code kan funktionerna hjälpa med autofyll och varningar vid fel inmatning om funktionen är kommenterad med JSDoc. [8]

5 Metod

5.1 Datainsamling och API-hantering

För att hämta tävlingsdata för säsongen 2026 användes JavaScripts funktionen fetch. Asynkrona funktioner med async/await valdes för att säkerställa att webbplatsen inte fryser medan data hämtas i bakgrunden. När anropet till **Jolpica F1 API** besvarats, omvandlades datan till JSON-format för att sedan filtrera ut och spara alla race från 2026 i en global variabel.

5.2 Kartor

För att skapa den interaktiva reseplaneraren kopplades koordinaterna (lon / lat) från Jolpica samman med **Leaflet**. Genom att använda koordinaterna från API-datan kunde en funktion skapas som dynamiskt centrerar kartan och placerar en markör på rätt plats för varje specifikt race. För att sedan rita själva kartan användes *tiles* från **OpenStreetMap** tillsammans med Leaflet-elementet.

5.3 Sök- och filtreringslogik

Sökfunktionen implementerades genom att använda en event-listener på användarens input i ett sökfält. En filter-funktion skapades som jämför användarens söksträng med fälten för "land" och banans "namn" för varje race i den redan hämtade datan. Detta gör att race-sektionen uppdateras dynamiskt utan att göra nya API anrop.

5.4 Implementering av ljust och mörkt läge

När användaren besöker på sidan för första gången läses användarens systeminställningar för prefers-color-scheme och sparar det i local storage. Beroende på värdet applicerades de korrekta färgerna. Utöver det lades en toggle-funktion som användaren manuellt kan använda, även det valet sparas i local storage så att inställningen kvarstår vid nästa besök.

6 Konstruktion

6.1 Automatiserat arbetsflöde med Vite

Projektet använder **Vite** som byggverktyg, vilket automatiserar flera processer under skapandet. Genom att använda Vites inbyggda live server appliceras kodändringar omedelbart i webbläsaren utan att sidan behöver laddas om manuellt. Vite sköter även optimering av koden genom exempelvis minifiering och konkatenering för att minimera laddningstider.

6.2 Modulär filstruktur

För att bibehålla en ren filstruktur är JavaScript-logiken uppdelad i specifika moduler med tydliga ansvarsområden där samtliga funktioner är dokumenterade med JSDoc:

page-handler.js

Hanterar Single Page Application (SPA)-logiken genom att dynamiskt rendera sidinnehåll.

race-data-handler.js

Sköter kommunikationen med Jolpica API, genererar race-kort dynamiskt och integrerar Leaflet-kartor i modalerna.

theme-handler.js

Sköter växlingen mellan ljust och mörkt läge samt lagrar användarens val i localStorage.

image-optimizer.js

En egenskriven modul som automatiserar hanteringen av responsiva bilder genom att generera <picture>-element med stöd för moderna format som AVIF och WebP. Med hjälp av tilläggen vite-image-optimizer och vite-imagetools skapas nya bilder i olika upplösningar och olika format samt att de komprimeras för förbättrad laddningstid. Funktionen returnerar en HTML-sträng. **(Se Figur 1)**

```
return `
  <picture>
    <source srcset="${avifSrcset}" sizes="${selectedSizeAlternative}" type="image/avif">
    <source srcset="${webpSrcset}" sizes="${selectedSizeAlternative}" type="image/webp">
    <img
      class="${className}"
      id="${id}"
      src="${fallback}"
      srcset="${jpgSrcset}"
      alt="${alt}"
      sizes="${selectedSizeAlternative}"
      loading="${lazy ? 'lazy' : 'eager'}"
      ${priority ? 'fetchpriority="high"' : ''}
      decoding="async"
    >
  </picture>
`;
}
```

Figur 1: <picture>-elementet som sträng.

Main.js

Fungerar som en startpunkt för webbsidans logik som importerar och initierar samtliga moduler.

6.3 Layout och styling med SCSS Partials

Stylingen är uppbyggd med **SCSS** och en modulär struktur av *partials*. Genom att separera exempelvis layoutlogik från variabler (som färger och teckensnitt) skapas en CSS-struktur som är lätt att underhålla.

Responsiviteten i race-sektionen hanteras via en **grid-layout** som dynamiskt anpassar antalet kolumner baserat på skärmbredd via *media queries* för specifika brytpunkter.

Genom att använda SCSS-variabler i kombination med ett dataset på :root kan hela webbplatsens färgtema skifta utan ändringar i kodens struktur. **(Se Figur 6 i kapitel 7.4)**

6.4 Filtrering och sökning

För att göra det enkelt för användaren att hitta specifika tävlingar implementerades en sökfunktion. Denna funktion sköter filtreringen av de tävlingar som redan har renderats på sidan, vilket gör sökningen mycket snabb då inga nya API-anrop krävs.

Processen fungerar enligt följande:

1. **Användarens input**: Funktionen läser in värdet från input-fältet och omvandlar det till små bokstäver med .toLowerCase(). På detta sätt blir sökningen "case-insensitive", vilket innebär att det inte spelar någon roll om användaren skriver med stora eller små bokstäver.

2. **Identifiering av sökbar data**: Genom att loopa igenom alla existerande race-containrar skapas en sökbar sträng för varje lopp. Denna sträng slår ihop land, loppets namn och banans namn till en enda sträng utan mellanslag.

3. **Visuell filtrering med CSS**: Funktionen kontrollerar om användarens sökord finns inuti den sökbara strängen med hjälp av .includes().
    a. Om det finns en matchning sätts elementets display-egenskap till flex så att det syns.
    b. Om sökordet inte matchar sätts display till none, vilket döljer loppet från användarens vy.

Att ändra CSS-egenskaper istället för att rendera nya objekt i DOM:en visade sig vara mycket mer responsivt än alternativet. Detta sparar datorkraft och ger användaren en omedelbar respons när de skriver i sökfältet. **(Se Figur 2)**

```
function searchData(dataArray) {
  const input = document.querySelector("#search-input").value.toLowerCase();
  const raceContainers = document.querySelectorAll('.race-container');

  raceContainers.forEach((container, index) => {
    const race = dataArray[index];
    const searchables = `${race.Circuit.Location.country} ${race.raceName}${race.Circuit.circuitName}`.toLowerCase()

    if (searchables.includes(input)) {
      container.style.display = "flex";
      if(container.dataset.theme === "open") {
        container.style.display = "grid";
      }
    }
    else {
      container.style.display = "none";
    }
  });
}
```

Figur 2: Funktion för att filtrera data.

6.5 Animationer och interaktioner

Animationerna är utformade för att ge visuell feedback och en responsiv känsla:

- **Hover-effekter**: Genom att använda CSS-transitioner på egenskapen transform skapas en mjuk skalning av race-korten när muspekaren förs över dem.

- **Modal-expansion**: Vid klick på ett kort beräknas dess position med getBoundingClientRect(). En klon av kortet expanderas därefter till sin bestämda storlek med hjälp av **requestAnimationFrame**. Eftersom elementet skapas dynamiskt och endast existerar temporärt i DOM:en krävdes JavaScript för att styra animationen. En teknik med nästlade requestAnimationFrame-anrop används för att säkerställa att webbläsaren hinner registrera elementets startläge innan animationen utförs, vilket resulterar i en fungerande animation. **(Se Figur 3)**

```
const rect = container.getBoundingClientRect();
const clone = container.cloneNode(true);
clone.id = 'clone';

requestAnimationFrame(() => {
      requestAnimationFrame(() => {
        clone.style.top =  '50%';
        clone.style.transform = 'translateY(-50%)';
        clone.style.left = standardPadding;
        clone.style.width = `calc(100vw - 2 * ${standardPadding} - ${scrollbarWidth})`;
        clone.style.minHeight = '70vh'
        clone.style.height = 'max-content';
        clone.style.maxHeight = `calc(100svh - 2 * ${standardPadding})`
        clone.style.boxShadow = 'none';
      });
    });
```

Figur 3: Skapande av klon och animering.

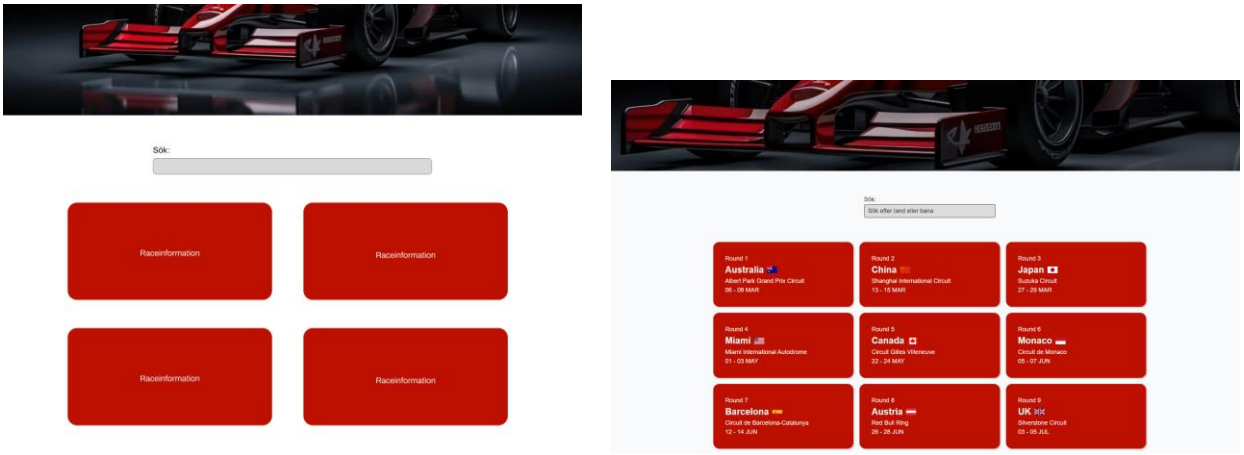
- **Fokus-hantering**: Vid öppen modal används attributet inert på bakomliggande element för att låsa tangentbordsfokus till modalen, vilket säkerställer god

tillgänglighet.

7 Resultat

7.1 Webbsidans gränssnitt

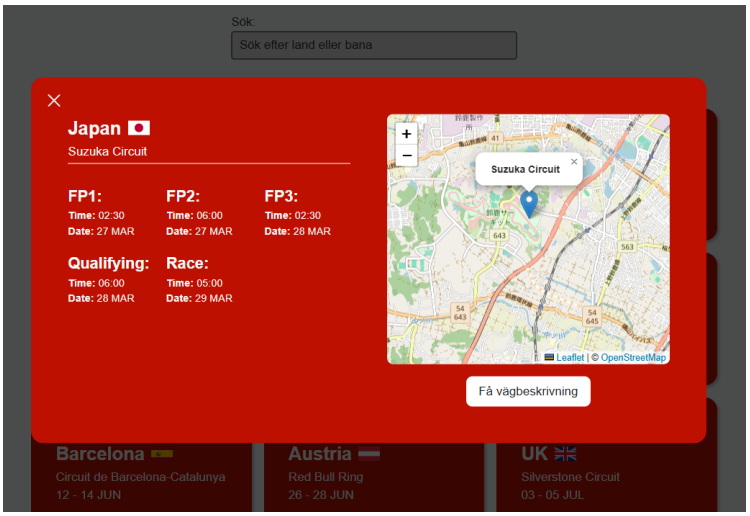
Den färdiga webbplatsen består av en responsiv layout där varje race presenteras med sitt eget kort. Dessa kort är klickbara vilket aktiverar en pop-up modal med mer ingående information samt kartan med markör för banans plats. Webbplats består endast av en landningssida med en enkel hero-sektion. Huvudfokuset ligger på informationen om racen. En enkel textruta ligger över korten för sökning. I den ursprungliga Figma-designen bestod kort-sektionen av två kolumner på breda skärmar. Under utveckling kändes det mer logiskt med 3 kolumner för de bredaste skärmarna. **(Se Figur 4)**



Figur 4: Jämförelse av layout (2 vs 3 kolumner).

7.2 Kartfunktioner

Varje lopp har en integrerad karta i sin tillhörande modal som visar korrekta koordinater. Kartorna är fullt interaktiva, vilket tillåter användaren att panorera och zooma. För att underlätta navigering har en knapp implementerats under varje karta som länkar användaren direkt till Google Maps vägbeskrivningar för den specifika destinationen. **(Se Figur 5)**



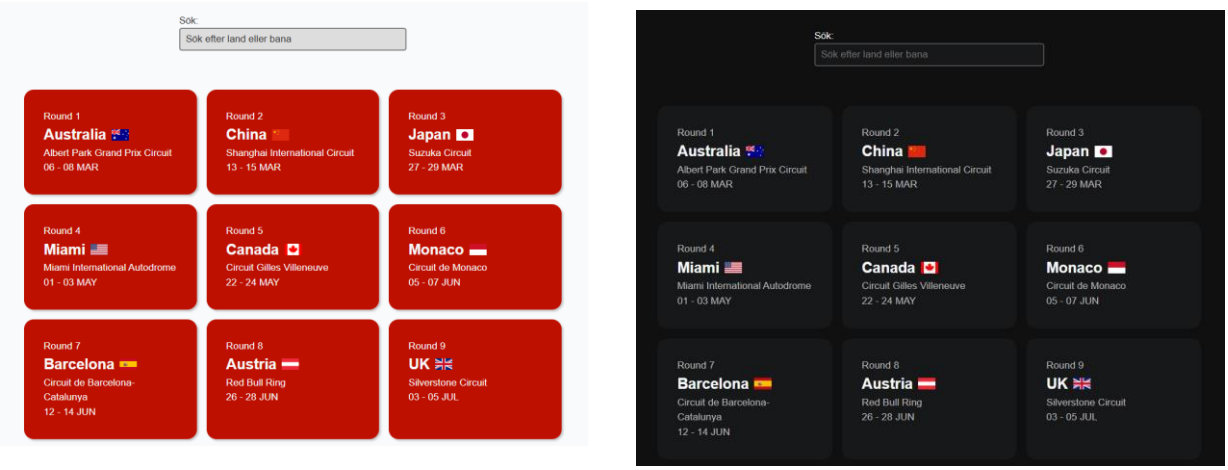
Figur 5: Vy av modal-fönstret med kartan och Google Maps-länken.

7.3 Sökning och filtrering i realtid

Implementeringen av sökfunktionen visade sig vara mycket användbar och responsiv. Användaren kan antingen skriva banans land eller namn och inforutorna sorteras ut dynamiskt utan någon fördröjning. Detta på grund av att data filtreras utan ytterligare nätverksanrop via fetch.

7.4 Visuella teman (Light/Dark mode)

Webbplatsen kan visas upp med två olika teman, ljust och mörkt. Inläsningen av användarens systeminställningar fungerar utan problem och att ändra temat via “tema-knappen” sparas korrekt i local storage. Det ljusa läget innehåller F1:s röda färg som huvudfärg och sidan kompletteras med andra ljusa färger. Det mörka temat ändrar alla färger till mörka nyanser av grått, blått och svart. **(Se Figur 6)**



Figur 6: Sida vid sida-jämförelse av ljust och mörkt tema.

7.5 Animationer och interaktioner

För att förbättra användarupplevelsen och ge webbsidan en modern känsla har flera subtila animationer implementerats:

- **Färgövergångar vid temabyten:** När användaren skiftar mellan ljust och mörkt läge sker inte färgändringen omedelbart. Istället används en mjuk CSS-transition på bakgrunds- och textfärger, vilket gör övergången behagligare för ögonen

- **Interaktiva kort (Hover-effekt):** För att tydliggöra att race-korten är klickbara har en hover-effekt lagts till. När muspekaren förs över ett kort sker en lätt skalning (förstoring) samtidigt som kortets rubrik blir understruken.

- **Modal-animering**: När en användare klickar på ett kort animeras modalen fram från det ursprungliga kortets position och tar upp en större del av skärmen. Denna effekt skapar en tydlig koppling mellan kortet och den fördjupade informationen, vilket gör att gränssnittet känns mer dynamiskt än om modulen dykt upp statiskt.

8 Slutsatser

8.1 Reflektion kring resultat och arbetsflöde

Målet med projektet var att bygga en modern sajt för F1-kalendern 2026, och jag är riktigt nöjd med hur det blev. Att använda **Vite** i projektet har varit en riktig uppgradering från tidigare projekt. Det har sparat mig massor av tid eftersom det sköter arbetsuppgifter som endast är mängdarbete, exempelvis att kompilera SCSS och optimera bilder, helt automatiskt. Det gjorde att jag kunde fokusera på att skriva kod istället för att sitta med monotona arbetsuppgifter.

Att jag valde att bygga webbplatsen som en **SPA (Single Page Application)** gjorde sluresultatet mycket mer professionellt. Det gör att sidan känns mer som en app än en gammal hemsida. Allt flyter på utan att skärmen blinkar till för att man byter sida, vilket gör den mycket trevligare att använda.

8.2 Lärdomar och utveckling

Innan det här projektet kände jag mig inte helt 100 på externa API:er, men efter att ha jobbat med flera olika API:er börjar det gå smidigare. Jag har lärt mig att hämta stora mängder data och sortera ut exakt det jag behöver, bland annat genom att koppla ihop koordinater med kartorna från Leaflet.

8.3 Framtida utveckling

Trots att webbsidan uppfyller alla uppställda krav finns det utrymme för vidareutveckling. Ett naturligt nästa steg vore att implementera statistik för resultat samt poängstatistik för förare och lag.

Sammanfattningsvis har det här projektet lärt mig hur mycket man vinner på att använda bra verktyg och ha en smart plan för hur koden ska struktureras från början.

9 Källförteckning

[1] Zorraquino “What is a mashup?”, https://www.zorraquino.com/en/dictionary/internet/what-is-mashup.html (Hämtad 2026-03-19)

[2] Chen, Michael, “What Is an API (Application Programming Interface)?”, *ORACLE,* https://www.oracle.com/se/cloud/cloud-native/api-management/what-is-api/ [2025-02-24] (Hämtad 2026-03-19)

[3] GitHub, “jolpica-f1”, https://github.com/jolpica/jolpica-f1 (Hämtad 2026-03-19)

[4] OpenStreetMap Wiki, “API”, https://wiki.openstreetmap.org/wiki/Sv:API [2022-10-05] (Hämtad 2026-03-19)

[5] Leaflet, “Overview”, https://leafletjs.com/ (Hämtad 2026-03-19)

[6] lukekarrys, “About npm”, npm Docs, https://docs.npmjs.com/about-npm [2023-10-23] (Hämtad 2026-03-19)

[7] VITE, “Getting Started”, https://vite.dev/guide/ (Hämtad 2026-03-19)

[8] Maddumage, Roshan, “A Comprehensive Guide to JSDoc Comments in JavaScript”, *Medium,* https://medium.com/@uomroshan/a-comprehensive-guide-to-jsdoc-comments-in-javascript-ed14df2351ef [2024-06-24] (Hämtad 2026-03-20)